# 1. Why Do We Need Delta Lake?

Parquet is already an efficient columnar format, so an important question is:

> If Parquet is good for analytics, why do we need Delta Lake?

Consider an orders pipeline where:

- New orders are inserted.
- Pending orders become completed.
- Incorrect records may need deletion.
- Multiple pipelines may read and write simultaneously.

Plain Parquet files do not provide built-in transactional `UPDATE`, `DELETE`, or `MERGE` operations by themselves.

Delta Lake adds **transactional reliability and table-management capabilities on top of Parquet data**. Class 07 — Delta Lake Fundament…

---

# 2. Parquet vs Delta Lake

## Plain Parquet

- Columnar file format
- Efficient compression
- Stores schema information
- No built-in table-level ACID transactions
- No built-in transactional `UPDATE`, `DELETE`, or `MERGE`
- No built-in table version history or Time Travel

## Delta Lake

- Uses Parquet files for actual data
- Adds a Delta transaction log
- Supports ACID transactions
- Supports `INSERT`, `UPDATE`, `DELETE`, and `MERGE`
- Maintains table version history
- Supports Time Travel subject to retention
- Provides schema enforcement

### Easy Way to Remember

Parquet
→ Efficient physical data storage

Delta Lake
→ Parquet data
+
Transaction Log
+
Transactional table capabilities

Class 07 — Delta Lake Fundament…

---

## 3. Delta Is Not Replacing Parquet

A common beginner misunderstanding is:

> Delta and Parquet are completely different file formats.

Instead, think:

Delta Table
↓
Parquet Data Files
+
_delta_log

The actual records are stored in Parquet files.

Delta adds metadata and transaction information that allows the system to understand the committed state of the table.

---

# 4. Inspecting a Delta Table

Suppose we already have:

```
SELECT *
FROM datax_db.orders
ORDER BY order_id;
```

Check its details:

DESCRIBE DETAIL datax_db.orders;

This can show information such as:

- Table format
- Physical location
- Other Delta table details

Remember:

Hive Metastore
→ Metadata about the table

Storage
→ Actual table data

---

# 5. What Is `_delta_log`?

Get the physical table location:

table_path = ( spark.sql("DESCRIBE DETAIL datax_db.orders") .select("location") .first()[0])print(table_path)

Inspect it:

display(dbutils.fs.ls(table_path))

You should see:

Parquet data files
_delta_log

The `_delta_log` is the **transaction log of the Delta table**.

It records information about committed table changes. Class 07 — Delta Lake Fundament…

---

## 6. What Does the Delta Log Track?

When the Delta table changes, Delta needs to know:

- Which files belong to the current table version
- Which files were added
- Which files are no longer part of the active table state
- Which transactions were committed

Example:

UPDATE happens
↓
Affected table data changes
↓
Files may be replaced or updated logically
↓
Transaction is recorded in _delta_log
↓
New committed table state

This does **not** mean every operation creates a full copy of the entire table.

Delta handles the files needed for the operation based on the table and operation.

---

# 7. Inside `_delta_log`

Inspect:

```
display(    dbutils.fs.ls(        f"{table_path}/_delta_log/"     ))
```

You may see:

- JSON commit files
- Checkpoint-related files

At this stage, remember:

### JSON Commit Files

Record transaction actions.

### Checkpoints

Help Delta reconstruct table state efficiently without replaying every historical JSON commit.

The main idea is:

> `_delta_log` provides the foundation for reliable transactions and table versioning.

Class 07 — Delta Lake Fundament…

---

# 8. What Are ACID Transactions?

ACID stands for:

A → Atomicity
C → Consistency
I → Isolation
D → Durability

These concepts help make table operations reliable.

---

## 9. Atomicity

Atomicity means:

> A transaction commits completely or does not commit.

Suppose a pipeline updates 1,000 records.

We do not want:

500 records updated
500 records failed

while still considering the transaction successful.

Delta commits the transaction atomically.

---

## 10. Consistency

Consistency means transactions preserve the consistency guarantees enforced by the system.

Delta can help enforce:

- Table schema
- Transaction consistency

But Delta does not automatically know every business rule.

For example:

quantity must always be > 0

still requires validation logic or appropriate constraints.

---

# 11. Isolation

Isolation matters when multiple processes operate simultaneously.

Example:

Pipeline A
→ Writing new orders

Pipeline B
→ Reading orders

Readers should see a **consistent committed snapshot**, not a partially written transaction.

---

# 12. Durability

Durability means:

> Once a transaction is successfully committed, its changes remain persisted.

Closing a notebook session does not undo a committed Delta transaction.

### Why ACID Matters

In production, multiple:

- Pipelines
- Jobs

- Users
- Queries

may interact with the same datasets.

Reliable transactions become essential. Class 07 — Delta Lake Fundament…

---

# 13. Create a Practice Delta Table

To avoid modifying the main orders table:

```
CREATE OR REPLACE TABLE datax_db.orders_delta_demo
USING DELTA
AS
SELECT *
FROM datax_db.orders;
```

This is:

CTAS
=
CREATE TABLE AS SELECT

The original table remains unchanged.

---

# 14. INSERT — Adding New Records

Suppose a new order arrives.

```
INSERT INTO datax_db.orders_delta_demo
(
   order_id,
   customer_id,
   product_id,
   order_date,
   quantity,
   unit_price,
   order_status
)
```

```
VALUES
(
    1011,
    'C108',
    'P101',
    '2026-09-06',
    1,
    899.00,
    'PENDING'
);
```

Verify:

```
SELECT *
FROM datax_db.orders_delta_demo
WHERE order_id = 1011;
```

The new record becomes part of the committed table state.

The transaction is tracked by Delta.

---

# 15. UPDATE — Modifying Existing Records

Suppose order `1011` moves from:

PENDING

to:

COMPLETED

Use:

```
UPDATE datax_db.orders_delta_demo
SET order_status = 'COMPLETED'
WHERE order_id = 1011;
```

Verify:

```
SELECT *
```

```
FROM datax_db.orders_delta_demo
WHERE order_id = 1011;
```

Delta handles the underlying transactional changes.

You do not manually open and rewrite Parquet files yourself. Class 07 — Delta Lake Fundament…

---

# 16. Be Careful with `UPDATE`

The `WHERE` condition is extremely important.

This:

```
UPDATE datax_db.orders_delta_demo
SET order_status = 'COMPLETED';
```

could affect many more records than intended.

In production:

- Verify the filter
- Validate target records
- Understand how many rows should be affected

before performing updates.

---

# 17. DELETE — Removing Records

Suppose order `1011` was only a test record.

```
DELETE FROM datax_db.orders_delta_demo
WHERE order_id = 1011;
```

Verify:

```
SELECT *
FROM datax_db.orders_delta_demo
WHERE order_id = 1011;
```

The current table state should no longer return that order.

---

# 18. Logical DELETE vs Physical Files

This is an important Delta concept.

Deleting a record logically does **not necessarily mean its old bytes are immediately removed from storage**.

Delta tracks which data belongs to the current table state.

Older files may remain for a retention period.

This becomes important for capabilities such as:

- Time Travel
- Historical table versions
- Later physical cleanup using `VACUUM`

Class 07 — Delta Lake Fundament…

---

# 19. DELETE vs Business Cancellation

These are not automatically the same thing.

Suppose an order is cancelled.

Business logic may require:

UPDATE order_status = 'CANCELLED'

rather than physically/logically deleting the order.

Why?

Because cancelled orders may still be needed for:

- Reporting
- Auditing
- Historical analysis

Whether to `UPDATE` or `DELETE` depends on the business requirement.

---

# 20. Delta Table History

After performing:

INSERT
UPDATE
DELETE

inspect history:

DESCRIBE HISTORY datax_db.orders_delta_demo;

You may see operations such as:

- `CREATE TABLE AS SELECT`
- `WRITE`
- `UPDATE`
- `DELETE`

and their corresponding table versions.

Every successful data-changing commit creates a new Delta table version. Class 07 — Delta Lake Fundament…

---

# 21. Table Versions

Conceptually:

Version 0
→ Initial table

Version 1
→ INSERT

Version 2
→ UPDATE

Version 3
→ DELETE

Exact version numbers can differ if you repeat operations.

This version history is what later enables concepts such as **Time Travel**.

---

# 22. Why Delta Matters in Real Pipelines

Imagine:

Pipeline 1
→ Inserts new orders

Pipeline 2
→ Updates order status

Analysts
→ Query the same table

We need:

- Reliable commits
- Consistent reads
- Table history
- Safe transactional changes

Delta helps provide these capabilities.

But it does not automatically solve every problem.

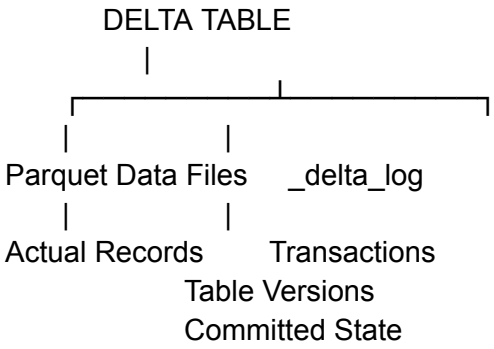
You still need to handle:

- Duplicates
- Business rules
- Data-quality validation
- Correct transformation logic

Class 07 — Delta Lake Fundament…

---

# 23. Complete Conceptual Flow




This is the core concept to remember.

---

# 24. Parquet vs Delta — Quick Revision

| Feature | Parquet | Delta |
|---|---|---|
| Columnar storage | Yes | Uses Parquet |
| Efficient compression | Yes | Yes |
| Schema information | Yes | Yes |
| Transaction log | No | Yes |
| ACID transactions | No | Yes |
| Transactional `UPDATE` | No | Yes |
| Transactional `DELETE` | No | Yes |
| MERGE support | No built-in transactional support | Yes |
| Table version history | No built-in | Yes |
| Time Travel | No built-in | Yes, subject to retention |

---

# 25. ACID — Quick Revision

Atomicity
→ All or nothing

Consistency
→ Preserve system-enforced consistency rules

Isolation
→ Concurrent users see consistent committed states

Durability
→ Committed data remains persisted

---

# 26. Commands to Remember

Create demo table:

```
CREATE OR REPLACE TABLE datax_db.orders_delta_demo
USING DELTA
AS
SELECT *
FROM datax_db.orders;
```

Insert:

```
INSERT INTO datax_db.orders_delta_demo
VALUES (...);
```

Update:

```
UPDATE datax_db.orders_delta_demo
SET ...
WHERE ...;
```

Delete:

```
DELETE FROM datax_db.orders_delta_demo
WHERE ...;
```

History:

```
DESCRIBE HISTORY datax_db.orders_delta_demo;
```

---

# 27. Common Mistakes

### Mistake 1: Thinking Delta Replaces Parquet Completely

Delta actually uses Parquet data files plus transaction metadata.

### Mistake 2: Thinking `_delta_log` Contains All Business Data

Actual records live in Parquet files.

The log tracks table transactions and state.

### Mistake 3: Updating Without a Proper `WHERE`

This can modify more records than intended.

### Mistake 4: Assuming DELETE Immediately Removes Old Physical Files

Logical deletion and physical cleanup are different concepts.

### Mistake 5: Assuming Delta Handles All Business Rules

Delta provides transactional guarantees, but business validations still need to be implemented.

---

# 28. Student Practice Assignment

Use:

datax_db.orders_delta_demo

### Task 1 — INSERT

Insert:

order_id → 1012
customer_id → C109
product_id → P103

order_date → 2026-09-07
quantity → 2
unit_price → 1499.00
order_status → PENDING

### Task 2 — UPDATE

Change:

1012
PENDING → COMPLETED

Verify using `SELECT`.

### Task 3 — DELETE

Delete order:

1012

Verify that it no longer exists in the current table.

### Task 4 — History

Run:

DESCRIBE HISTORY datax_db.orders_delta_demo;

Identify:

- INSERT
- UPDATE
- DELETE
- Corresponding table versions

Class 07 — Delta Lake Fundament…

---

# 29. Interview-Relevant Questions

### 1. What is Delta Lake?

Delta Lake is a storage layer that uses Parquet data files together with a transaction log to provide reliable transactional table capabilities.

## 2. Does Delta store data in a completely different format from Parquet?

No.

Delta uses Parquet files for actual data storage and adds the Delta transaction log.

## 3. What is `_delta_log`?

It is the transaction log that tracks committed table changes and helps Delta determine the state of each table version.

## 4. What does ACID stand for?

- Atomicity
- Consistency
- Isolation
- Durability

## 5. Why can Delta support transactional `UPDATE` and `DELETE` while plain Parquet alone cannot?

Parquet is primarily a file format.

Delta adds a transaction log and commit protocol that tracks active files, committed changes, and consistent table snapshots. Class 07 — Delta Lake Fundament…

## 6. What is Atomicity?

A transaction either commits completely or does not commit.

## 7. What is Isolation?

Readers and writers can interact while users see consistent committed table states rather than partial transactions.

## 8. What is Durability?

Once a transaction commits successfully, its changes remain persisted.

## 9. Does Delta automatically enforce all business rules?

No.

Business validations still need to be implemented separately.

### 10. Why is `WHERE` important in an `UPDATE` or `DELETE`?

Because an incorrect or missing filter can affect more records than intended.

### 11. Does deleting a Delta record immediately remove its historical physical data?

Not necessarily.

Older files may remain for a retention period.

### 12. What does `DESCRIBE HISTORY` show?

It shows the history of operations and versions committed to a Delta table.

---

# 30. Key Takeaway

Remember the relationship:

Parquet
↓
Efficient Storage

Parquet
+
_delta_log
↓
Delta Lake
↓
ACID Transactions
↓
INSERT / UPDATE / DELETE
↓
Reliable Table Versions

The most important idea from this class is:

**Delta Lake keeps Parquet's efficient storage while adding transactional reliability and version-aware table management.**